

Planned experiments: prior recovery on held-out test data

Working note for the SIAP revision numerics (companion to `convergence_work.tex`)

GPL

August 25, 2026

This note records two experiments approved for execution. Experiment 1 recomputes the reported error tables as relative quantities, without retraining. Experiment 2 is the main experiment: on a held-out test set, we compare the paper’s two-network recovery against a one-network recovery that uses the exact proximal map, at matched data, test set, and training budget. The convergence theory of `convergence_work.tex` is the reference; cross-references below are to its results by name.

1 Setting and notation

We keep the notation of `convergence_work.tex`. Fix $t > 0$ and a prior J on $\mathbb{R}^n$. Write $\psi(\mathbf{x}) = \frac{1}{2} \|\mathbf{x}\|_2^2 - t S(\mathbf{x}, t)$ for the convex potential and $g = \psi^*$ for its conjugate, so that

$$\nabla\psi = \text{prox}_{tJ}, \quad g(\mathbf{y}) = t J_{\text{BVS}}(\mathbf{y}) + \frac{1}{2} \|\mathbf{y}\|_2^2, \quad \nabla g = (\nabla\psi)^{-1}, \quad (1)$$

where J_{BVS} is the convex prior the data actually determine ($J_{\text{BVS}} = J$ when J is convex; the two differ only where J is nonconvex, as for $J = -\|\cdot\|_1$). The sampling identities give conjugate pairs $(\mathbf{y}_k, g(\mathbf{y}_k))$ with $\mathbf{y}_k = \nabla\psi(\mathbf{x}_k) = \text{prox}_{tJ}(\mathbf{x}_k)$ and $\mathbf{x}_k = \nabla g(\mathbf{y}_k)$, and $g(\mathbf{y}_k) = \langle \mathbf{x}_k, \mathbf{y}_k \rangle - \psi(\mathbf{x}_k)$.

A single learned potential appears throughout: a convex network G_ϕ fit to samples of g . From it we read the two recovered objects,

$$\text{recovered prior } \hat{J} = G_\phi - \frac{1}{2} \|\cdot\|_2^2 \approx J_{\text{BVS}}, \quad \text{recovered inverse prox } \nabla G_\phi \approx (\nabla\psi)^{-1} = \nabla g. \quad (2)$$

Both are evaluated only on the query box $[-4, 4]^n$, where all errors are reported.

2 Experiment R: relative-error renormalization

Experiment 1 (relative errors; no retraining). Recompute the reported errors in the four tables as relative L^2 quantities. For the prior, report $\|\hat{J} - J_{\text{BVS}}\|_2 / \|J_{\text{BVS}}\|_2$ over the shared evaluation set; for the proximal map, report $\|\widehat{\text{prox}} - \text{prox}_{tJ}\|_2 / \|\text{prox}_{tJ}\|_2$. Both follow from the stored run metrics and closed-form ground truth on the evaluation points, with no training. The proximal column requires one forward pass of the saved networks, since only the prox residual, not the prox error against ground truth, is stored.

The reported tables carry raw mean-squared error, while ψ and g grow in magnitude with n (the potential is essentially $\frac{1}{2} \|\cdot\|_2^2$, of size $O(n)$ on a fixed box), so a fixed relative error produces a raw error that grows like n^2 . Experiment 1 removes that scale and fixes the baseline against which every later claim of “works” or “fails” is judged.

3 Experiment C: one network vs two networks on held-out test data

The paper recovers the prior with two networks in sequence: a first network learns ψ , and its gradient generates the conjugate samples on which a second network is trained. For all four families the proximal map is available in closed form — soft-thresholding for $\pm \|\cdot\|_1$, a linear map for the concave quadratic, and a branch map for the min-plus mixture at $\sigma_1 = \sigma_2$. The exact map lets us build the conjugate samples directly and train a *single* network, deleting the first one. This is the plug-and-play scenario, in which a denoiser supplies the proximal map and one only fits its potential. Experiment 2 asks the single quantitative question: does the one-network recovery match, or beat, the paper’s two-network recovery, judged on data neither network saw?

Experiment 2 (matched comparison of the two recoveries). For every family and dimension, generate one set of training inputs and one held-out test set, then run both methods on them and report the test accuracy of each.

Data (shared). Training inputs $\mathbf{x}_k$, $k = 1, \dots, N$, drawn uniformly on the training box $[-A, A]^n$ (seed 1), with $A = \text{train_halfwidth}(4)$ chosen so that $\nabla\psi$ maps the training box onto the query box; a validation set (seed 2) on the same box; and a held-out test set $\{\mathbf{z}_j\}_{j=1}^{1000}$ drawn on the query box $[-4, 4]^n$ (seed 3). The test set is disjoint from training and validation and is used by neither method during training.

Method 1 (one network, exact prox). Using the closed-form prox, form the exact pairs $(\mathbf{y}_k, g(\mathbf{y}_k))$ with $\mathbf{y}_k = \text{prox}_{tJ}(\mathbf{x}_k)$ and $g(\mathbf{y}_k) = \langle \mathbf{x}_k, \mathbf{y}_k \rangle - \psi(\mathbf{x}_k)$, and train one network G_ϕ on them by value MSE.

Method 2 (two networks, the paper). Train a first network ψ_θ on $\{(\mathbf{x}_k, \psi(\mathbf{x}_k))\}$ by value MSE; form the conjugate pairs $(\nabla\psi_\theta(\mathbf{x}_k), \langle \cdot \rangle - \psi_\theta(\mathbf{x}_k))$ that the first network emits; train a second network G_ϕ on them by value MSE.

Every network — G_ϕ in Method 1, and ψ_θ and G_ϕ in Method 2 — uses the same architecture, the same loss, and the same budget of S optimizer steps. The *only* difference between the two methods is the source of the second network’s training targets: the exact prox (Method 1) or the first network’s output (Method 2). On the held-out test set report, for each method, the two relative L^2 errors of eq. (2):

$$\text{prior accuracy } \frac{\|\hat{J} - J_{\text{BVS}}\|_2}{\|J_{\text{BVS}}\|_2}, \quad \text{inverse-prox accuracy } \frac{\|\nabla G_\phi - (\nabla\psi)^{-1}\|_2}{\|(\nabla\psi)^{-1}\|_2}, \quad (3)$$

with J_{BVS} and $(\nabla\psi)^{-1}$ evaluated in closed form at the test points.

3.1 Why the comparison is correct and fair

The experiment compares the two methods on identical terms, and it compares the right quantities.

Same recovered objects, by the same formulas. Both methods produce a single conjugate network G_ϕ , and from it the same prior $\hat{J} = G_\phi - \frac{1}{2} \|\cdot\|_2^2$ and the same inverse prox ∇G_ϕ by the identical rules of eq. (2). We never compare a prior against a proximal map, or one method’s

forward map against the other’s inverse map; each panel is one quantity against one closed-form reference.

Same data, same test, same budget. Both methods train on the same inputs $\mathbf{x}_k$ and are scored on the same held-out points $\mathbf{z}_j$, so the numbers measure generalization, not memorization, and no method sees the test set. Each network is trained for the same S steps with the same architecture and loss, so neither the second network of Method 1 nor that of Method 2 is handicapped. Method 2 additionally pays to train its first network; that extra cost is a property of the two-network method, not an artifact of the comparison, and the point of the experiment is precisely to ask whether that cost buys anything on held-out data.

The single isolated difference is the target source. Method 2’s second network is trained on conjugate pairs corrupted by the first network’s gradient error; Method 1’s is trained on exact pairs. Everything else is held fixed, so any gap in test accuracy is attributable to that corruption alone. The two methods’ training inputs to G_ϕ have the same functional form, $\nabla\psi(\mathbf{x}_k)$ versus $\nabla\psi_\theta(\mathbf{x}_k)$; their difference is exactly the effect under study, and both networks’ training support covers the query box by the choice of A .

The reference is the recoverable object. The prior is scored against J_{BVS} , the convex prior the data determine, which equals J for the three convex families and is the convex envelope for $J = -\|\cdot\|_1$. Both methods are scored against the same J_{BVS} , so the nonconvex gap $J - J_{\text{BVS}}$ affects them equally and never distorts the comparison.

Remark (a structural prediction). Three of the four families are separable ($\pm\|\cdot\|_1$ and the concave quadratic), so under the exact prox their recovery factors into n one-dimensional problems and the relative error should be nearly independent of n . The min-plus mixture with $\mu_2 = 1/\sqrt{n}$ is the only non-separable family. Experiment 2 shows whether the one-network recovery preserves that dimension-independence that the two-network cascade may lose.

4 Execution plan

The two experiments run without touching any reported module, so the four family notebooks, `bin/plot.py`, and `bin/run_all.py` are unaffected and their cached metrics still load. The guiding principles are the same five as before: add code rather than edit shared code; compose already-tested building blocks; isolate outputs in a separate namespace; make the driver idempotent so an interrupted run resumes; and gate cost behind a staged test ladder.

4.1 Shared, tested building blocks

Method 2 is the reported pipeline, so it reuses its exact functions: `src/train.py:train_potential` (value MSE), `src/recovery.py:conjugate_samples` (the conjugate pairs a trained ψ_θ emits), and `src/recovery.py:evaluate_learned_prior_G`. Method 1 reuses `train_potential` on the exact pairs. The only new mathematics is the closed-form forward prox $\nabla\psi$ and the exact conjugate values, in `src/exact_prox.py`; it is pure and unit-tested, and it is the analytic inverse of the existing `Problem.preimage`.

4.2 New code, additive only

- `src/exact_prox.py` — pure helpers: the forward prox $\nabla\psi$ per family, the exact triples $(\mathbf{y}_k, g(\mathbf{y}_k), \mathbf{x}_k)$, the closed-form J_{BVS} and inverse prox at arbitrary points, and the relative- L^2 reducers. Unit-testable in isolation.

- `bin/compare_recovery.py` — the Experiment 2 driver: for a (family, dim) it builds the shared data, runs Method 1 and Method 2 at a common budget S , scores both on the held-out test set, and writes one metrics JSON. Flags mirror `run_all.py`: `--families`, `--dims`, `--steps`, `--allow-high-dim`, `--out`.
- `bin/relative_errors.py` — the Experiment 1 driver: read each reported `logs/<run>metrics.json`, recompute the relative value and proximal errors against closed-form ground truth, and write `logs/summary_relerr.csv`.
- `bin/plot_compare.py` — render the Experiment 2 figure: two rows (prior accuracy, inverse-prox accuracy), one column per family, two lines per panel (one network, two networks), against dimension. A budget note appears while $S < 250000$.

4.3 Isolation, idempotence, and namespace

Outputs live in a separate namespace: `logs/<family>_<dim>D_compare_metrics.json`, an aggregate `logs/summary_compare.csv`, and `logs/summary_relerr.csv`. These match the `*metrics.json` and `summary*.csv` patterns `.gitignore` already tracks. Diagnostic checkpoints go to an ignored `logs/compare_ckpt/`. The driver is resumable: each (family, dim) unit writes one result file with a terminal `"status": "done"` sentinel and is skipped on restart. Dimensions above eight are gated behind `--allow-high-dim`, per the standing rule against unprompted high-dimensional runs.

4.4 Staged test ladder

- (T1) *Unit*, seconds, no training: `tests/test_exact_prox.py` checks that $\nabla\psi$ is the gradient of ψ to machine precision, that the exact triples satisfy $g(\mathbf{y}_k) = \langle \mathbf{x}_k, \mathbf{y}_k \rangle - \psi(\mathbf{x}_k)$, and that the relative-error reducer matches a hand computation.
- (T2) *Smoke*, one to two minutes: `bin/compare_recovery.py --smoke` runs one family at $n = 2$ with a reduced budget, exercising both methods end to end.
- (T3) *Low-dimension, matched budget*, a few hours: all four families at $n \in \{2, 4, 8\}$, both methods at the same S .
- (T4) *High-dimension, opt-in*, overnight: $n \in \{16, 32, 64\}$ behind `--allow-high-dim`, resumable.

Both methods are trained fresh at the same budget in every stage; no method is ever read from a previously trained checkpoint at a different budget.

5 Deliverables

Experiments 1 and 2 write their numbers to tracked JSON under `logs/`, mirroring the existing metrics files. The comparison figure is reproduced by `bin/plot_compare.py` from cached metrics. No result in this note supersedes a reported table until reviewed.